

```

        # If enough time has elapsed, move on to the next image
        if self.elapsed > self.durationPerImage:
            self.index = self.index + 1

            if self.index < self.nImages: # move on to next image
                self.imageStartTime = time.time()

            else: # animation is finished
                self.playing = False
                self.index = 0 # reset to the beginning

    def draw(self):
        # Assumes that self.index has been set earlier - in the update() method.
        # It is used as the index into the imagesList to find the current image.
        theImage = self.imagesList[self.index] # choose the image to show

        self.window.blit(theImage, self.loc) # show it

```

Listing 14-3: The SimpleSpriteSheetAnimation class

This class is very similar to `SimpleAnimation`, but because this animation is based on a sprite sheet, the `__init__()` method must be passed different information ❶. The method requires the standard `window` and `loc` parameters, as well as:

- imagePath** A path to a sprite sheet image (single file)
- nImages** The number of images in the sprite sheet
- width** The width of each subimage
- height** The height of each subimage
- durationPerImage** How long (in seconds) to show each image

Given these values, the `__init__()` method loads the sprite sheet file, and it uses a loop to split up the larger image into a list of smaller subimages through a call to the pygame `subsurface()` method. The smaller images are then appended into the `self.imagesList` list for use by the other methods. The `__init__()` method uses a counter to count the number of subimages, up to the number specified by the caller; therefore, the last row of images does not need to be a full row. For example, we could have used a sprite sheet